

A Technical Report on

Recursive Language Models: A paradigm shift in scaling inference, challenging traditional RAG

submitted in partial fulfillment of the requirements
for the award of the degree of

Bachelor of Engineering
in
Artificial Intelligence and Data Science

By
Valleboina Shiva Prasad (1601-22-771-063)

Under the supervision of

Dr. K. Radhika
Professor



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE
CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
(AUTONOMOUS)

(An Autonomous Institution, Affiliated to Osmania University, Approved by AICTE, Accredited by NAAC with A++ Grade and Programs Accredited by NBA)

Chaitanya Bharathi Post, Gandipet, Kokapet(Vill), Hyderabad, Ranga Reddy - 500075, Telangana

www.cbit.ac.in

APRIL 2026



**CHAITANYA BHARATHI
INSTITUTE OF TECHNOLOGY**
An Autonomous Institute | Affiliated to Osmania University
Kokapet Village, Gandipet Mandal, Hyderabad, Telangana-500075, www.cb.it.ac.in

Approved by Affiliated to UGC Autonomous 12 Programs Accredited by Grade A++ in All India Ranking 151-200 Band Certified by

      

COMMITTED TO
RESEARCH,
INNOVATION AND
EDUCATION

47
years

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

DEPARTMENT CERTIFICATE

This is to certify that the Technical report entitled **Recursive Language Models: A paradigm shift in scaling inference, challenging traditional RAG** being submitted by **Valleboina Shiva Prasad**, bearing **H.T.No: 1601-22-771-063**, in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in **Artificial Intelligence and Data Science**, is a record of bonafide work carried out by him, during the academic year 2025–2026.

Faculty Coordinator

Head of the Department

ACKNOWLEDGEMENT

The successful completion of this Technical Seminar and report would not have been possible without the guidance and support of several individuals.

I would like to express my sincere gratitude to **Dr. K. Radhika**, Professor, **Dr. T. Satya Kiranmai**, Assistant Professor, **Dr. S. Shobarani**, Assistant Professor, Department of Artificial Intelligence and Data Science, for their invaluable guidance, constructive feedback, and unwavering support throughout the course of this Technical Seminar. Their expertise and mentorship were pivotal in shaping this technical report.

My heartfelt appreciation goes to **Dr. K. Radhika**, the Head of the Department, Department of Artificial Intelligence and Data Science, for granting the opportunity to undertake this Technical Seminar and providing access to departmental facilities.

I also extend my gratitude to the faculty and staff members of the Department of Artificial Intelligence and Data Science, as well as my parents and friends, for their continuous support and encouragement throughout this journey.

ABSTRACT

The rapid evolution of large language models (LLMs) has introduced a fundamental bottleneck: the inability to process and reason over arbitrarily long contexts. Standard attention mechanisms suffer from probability mass dilution and context rot as input lengths scale to millions of tokens, while retrieval-augmented generation (RAG) systems sacrifice global context through sparse, heuristic-based retrieval. This work presents the Recursive Language Model (RLM) paradigm, a structural framework that redefines long-context reasoning by shifting from passive token ingestion to active programmatic navigation. Unlike conventional scaffolds that flood the context window, RLMs externalize the entire prompt into a persistent Read-Eval-Print Loop (REPL) environment, treating documents as interactive digital worlds to be explored with code. Through symbolic recursion, the root model orchestrates programmatic sub-calls for dense semantic reasoning, maintaining a clean context window while enabling systematic traversal of multi-million-token corpora. Empirical evaluation across rigorous benchmarks BrowseComp-Plus (6M–11M tokens), OOLONG-Pairs ($O(N^2)$ relational reasoning), and LongBench-v2 CodeQA demonstrates that RLMs achieve up to 91.3% accuracy, significantly outperforming RAG-based and summary-based baselines. The framework transforms memory bottlenecks into inference-time scaling, with median API costs comparable to standard model calls. This work establishes that the fundamental limitation of LLMs is not context window size but the software scaffold surrounding them, positioning RLMs as a foundational paradigm for high-stakes domains requiring deep, reliable reasoning over massive datasets.

Keywords : Recursive Language Models, Long-context reasoning, Programmatic navigation, Inference-time scaling, REPL environment, Symbolic recursion, Large Language Models

LIST OF TABLES

2.1	Comparative Analysis of Long-Context Reasoning Frameworks	5
-----	---	---

LIST OF FIGURES

TABLE OF CONTENTS

Contents	Page No.
DEPARTMENT CERTIFICATE	i
ACKNOWLEDGEMENT	ii
ABSTRACT	iii
LIST OF TABLES	iv
LIST OF FIGURES	v
TABLE OF CONTENTS	vi
CHAPTER - 1 : INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Scope	2
1.5 Applications	2
1.6 Alignment with Sustainable Development Goals (SDGs)	2
CHAPTER - 2 : LITERATURE SURVEY	3
2.1 Comparative Analysis of Long-Context Reasoning Frameworks	3
CHAPTER - 3 : METHODOLOGY	6
3.1 The Core Insight: An Environment Shift	6
3.2 The Persistent REPL Environment (The Sandbox)	6
3.3 The Engine of Discovery: Symbolic Recursion	6
3.4 Adaptive Recursion vs. Static Chunking	7
3.5 The Three Fundamental Constraints of RLMs	7
3.6 Algorithm 1: The RLM Loop (The Right Way)	7
3.7 Algorithm 2: Standard Scaffolds (The Wrong Way)	8
3.8 RLM vs. AI Agents: A Crucial Distinction	8
3.9 Conclusion of the Methodology	8

3.10	Evaluation Benchmarks and Testing Setup	9
3.10.1	Understanding Task Complexity Scaling	9
3.10.2	Benchmark 1: BrowseComp-Plus (The 1K Document Challenge)	9
3.10.3	Benchmarks 2 & 3: The OOLONG Family (Information-Dense Reasoning)	9
3.10.4	Benchmark 4: LongBench-v2 CodeQA	9
3.10.5	Testing Models and Baseline Configuration	10
CHAPTER - 4 : ANALYSIS AND DISCUSSION		11
4.1	Experimental Baselines	11
4.1.1	Baseline 1: CodeAct (+ BM25 Retriever)	11
4.1.2	Baseline 2: CodeAct with Sub-calls	11
4.1.3	Baseline 3: The Summary Agent Baseline	11
4.1.4	Conclusion of the Baseline Framework	12
4.2	Experimental Results & Analysis	12
4.2.1	Observation 1: Massive Scale and Performance Dominance . .	12
4.2.2	Observation 2: REPL vs. Recursive Sub-Calling (An Ablation Analysis)	12
4.2.3	Observation 3: Scaling Curves & Stability	12
4.2.4	Observation 4: Inference Cost Transformation	13
4.2.5	Observation 5: Model Agnosticism & Behavioral Differences	13
4.2.6	Observation 6: Fine-Tuning Native RLMs (Bootstrapping) . .	13
4.3	Cost, Latency, & Performance Dynamics	13
4.3.1	Cost Distributions & The High Variance Long Tail	13
4.3.2	Runtime Limitations and the Latency Tradeoff	14
4.3.3	The Path to Asynchronous Execution	14
4.4	Emergent Patterns (Case Studies)	14
4.4.1	Pattern 1: Filtering Input Information via Prior Knowledge . .	14
4.4.2	Pattern 2: Recursive Delegation and Semantic Batching	14
4.4.3	Pattern 3: Bypassing Output Limits (The Final Answer Variable)	14
4.4.4	Case Study Failures: The Sub-Call Trap and Over-Verification	15
CHAPTER - 5 : CONCLUSION		16
5.1	Key Learnings and Summary of Findings	16
5.1.1	The Latency Nuance: Why RAG Survives	16
5.1.2	The Heuristic Reality and The "Wait-To-Think" Economy . .	16
5.1.3	Future Hybridization: The Best of Both Worlds	16
5.2	Importance of the Topic	17

5.3	Future Scope	17
5.3.1	Bootstrapping Reasoning and The Training Recipe	17
5.3.2	A New Axis of Scaling	17
5.3.3	Architectural Missteps: What Didn't Work	17
5.3.4	Parsing Fragility and The Sub-Call Trap	18
5.3.5	Latency and Depth Constraints	18
REFERENCES		19

CHAPTER 1

INTRODUCTION

1.1 Background

The field of Artificial Intelligence has undergone a rapid transformation, moving from simple prompt-response interactions to complex, long-horizon tasks. Modern AI systems must now process continuous workflows involving tens to hundreds of millions of tokens, with reasoning models becoming the primary interface to Large Language Models (LLMs) [1]. This shift goes beyond the capabilities of standard Transformers [2].

A critical industry challenge is scaling the effective context size of these models by orders of magnitude, primarily using inference-time compute rather than infinitely expanding pre-training windows. However, feeding larger token strings into neural networks is not sustainable for true long-context understanding [1].

1.2 Problem Statement

The primary bottleneck is the inherent flaw of the attention mechanism, which is approaching a mathematical barrier. The issue is **Probability Mass Dilution**: as input sequence length grows, the model's attention must distribute across all tokens, causing the probability mass for any single token to decrease. At a 1-million-token scale, critical information (the "signal") gets drowned out by irrelevant "noise," hindering information retrieval and reasoning [2, 3].

Historically, the "Needle in a Haystack" test validated context windows but is now saturated and flawed. It relies on simple keyword matching for isolated facts, which does not reflect real-world complexity involving active Q&A sessions requiring deep semantic understanding over entire corpora [3]. This leads to **Context Rot**, where a model's logical reasoning degrades as context length increases, regardless of its advertised window size, severely impacting enterprise efficiency and cost [4].

The industry's primary workaround for this bottleneck has been **Retrieval-Augmented Generation (RAG)** [5]. RAG stores information externally and retrieves relevant chunks via vector search, bypassing context limits. However, it is a patch. The "Glued Engine" problem separates the search from the reasoning engine, blinding it to global context. It also suffers from a bias towards **sparse retrieval**, failing at dense semantic aggregation tasks where answers require synthesizing information

from across the entire dataset [6].

1.3 Objectives

The objective of this work is to explore and evaluate the Recursive Language Model (RLM) paradigm as a solution to the limitations of current long-context processing methods. This involves understanding its core principles, architecture, and performance against established baselines.

1.4 Scope

This study focuses on the theoretical underpinnings and empirical evaluation of RLMs for processing massive contexts (millions of tokens). It compares RLM performance against standard RAG, summary agents, and other agentic baselines using a set of complex benchmarks designed for multi-hop reasoning and dense information aggregation.

1.5 Applications

The RLM paradigm has significant applications in areas requiring deep, accurate reasoning over extensive datasets, such as legal document analysis, medical literature review, complex codebase understanding, and advanced scientific research.

1.6 Alignment with Sustainable Development Goals (SDGs)

This work aligns with several United Nations Sustainable Development Goals:

- **SDG 4: Quality Education:** By improving AI’s ability to process and reason over vast educational materials, RLMs can power advanced tutoring systems and educational tools that provide deeper, more accurate understanding for learners.
- **SDG 9: Industry, Innovation, and Infrastructure:** RLMs represent a significant innovation in AI architecture, enabling the development of more robust and capable infrastructure for knowledge management, research, and development across industries.
- **SDG 12: Responsible Consumption and Production:** By enabling more efficient, selective processing of information (rather than brute-force ingestion), RLMs can lead to more computationally efficient AI systems, reducing energy consumption and promoting responsible resource use.

CHAPTER 2

LITERATURE SURVEY

2.1 Comparative Analysis of Long-Context Reasoning Frameworks

An extensive review of contemporary literature reveals diverse strategies for overcoming LLM limitations in processing extensive datasets. While the focus has been on optimizing retrieval mechanisms, the demand for deep semantic reasoning over long-horizon tasks is exposing the limitations of static retrieval paradigms. The following provides a comparative analysis of prominent frameworks.

Hybrid Retrieval Generation for Structured Reasoning in LLMs (Springer Nature, 2026)

This research proposes a hybrid RAG architecture that integrates knowledge graph retrieval with dense vector embeddings to enhance multi-hop inference and explainability [7]. While this improves accuracy and transparency, the dual-retrieval pipeline increases system complexity, requires domain-specific tuning, and incurs higher computational cost compared to unimodal RAG systems. Unlike emerging recursive paradigms, this static hybrid approach remains bottlenecked by its rigid overhead.

Collaborative Reasoning Framework for Long-Context Q&A (Elsevier, 2026)

This study addresses the "lost-in-the-middle" phenomenon [3] by introducing a collaborative reasoning framework that integrates RAG with multi-perspective logic [8]. Retrieved chunks are cross-validated, improving long-context understanding and accuracy. However, its effectiveness is still bottlenecked by the initial retrieval phase and shows limited scalability for extremely large corpora, failing to match the adaptive exploration of programmatic frameworks.

TreeQA: Logic Tree-Based RAG for Multi-Hop QA (Elsevier, 2025)

The TreeQA framework handles multi-hop QA by decomposing queries into a logic tree of sub-questions for iterative retrieval and validation [9]. This enhances interpretability and factual accuracy. However, it introduces substantial latency due to its iterative, blocked reasoning process. While structurally similar to recursive

sub-calling, its reliance on a rigid pre-computed logic tree prevents dynamic, on-the-fly exploration.

RAG + Code Interpreter Framework for Educational QA (PLOS ONE, 2025)

This framework augments RAG with an active code interpreter to solve queries needing both textual knowledge and computational logic, effectively eliminating math-based hallucinations in specific domains [10]. However, it is highly domain-specific, struggles with open-domain tasks, and its performance depends on the stability of tool integration. It uses code as a calculator, not a navigational tool for large contexts.

Enhanced Retrieval Scheme using Probabilistic + Semantic Association (MDPI, 2024)

This scheme fuses probabilistic relevance models with neural semantic association to improve baseline retrieval accuracy for LLM-based QA [11]. While it patches basic retrieval shortfalls, it offers limited support for complex multi-hop reasoning and lacks deep reasoning integration, highlighting an over-reliance on optimizing search heuristics rather than the underlying reasoning scaffold.

Summary of Literature Survey

The following table summarizes the key characteristics, findings, and limitations of the surveyed frameworks.

Table 2.1: Comparative Analysis of Long-Context Reasoning Frameworks

Name	Published by & Year	Key Findings	Limitations
Hybrid Retrieval Generation for Structured Reasoning	Springer (Springer Nature) 2026	Proposed hybrid RAG combining graph retrieval + dense embedding for structured reasoning improves multi-hop inference and explainability in LLM systems [7]	Increased system complexity, requires careful tuning of hybrid retrieval components, higher computational cost.
Collaboration Reasoning Framework for Long-Context Q&A	Elsevier (Expert Systems with Applications) 2026	Addresses "lost-in-the-middle" problem, integrates RAG with collaborative reasoning to improve long-context understanding and answer accuracy [8]	Still depends on external retrieval quality, limited scalability for extremely large corpora.
TreeQA Logic Tree-Based RAG for Multi-Hop QA	Elsevier (Knowledge-Based Systems) 2025	Introduces hierarchical logic-tree decomposition with iterative validation, significantly improves interpretability and factual accuracy in multi-hop QA [9]	Higher latency due to iterative reasoning; complex pipeline design.
RAG + Code Interpreter Framework for Educational QA	PLOS ONE (Public Library of Science) 2025	Combines RAG with code interpreters to handle logical + computational reasoning tasks; enhances correctness in domain-specific QA [10]	Domain-specific performance depends on tool integration; not generalizable across all tasks.
Enhanced Retrieval Scheme using Probabilistic Semantic Association	MDPI (Applied Sciences) 2024	Improves retrieval accuracy by combining probabilistic relevance with semantic similarity; enhances LLM-based QA systems [11]	Limited support for multi-hop reasoning; lacks deep reasoning integration.

CHAPTER 3

METHODOLOGY

3.1 The Core Insight: An Environment Shift

The fundamental challenge in modern AI is not data volume alone, but how models process it. Standard attention mechanisms degrade with massive contexts, and RAG strips away holistic understanding [2, 5]. To overcome these limitations, Recursive Language Models (RLMs) introduce a paradigm shift, moving from brute-force token ingestion to programmatic, algorithmic exploration [1].

The RLM paradigm’s core insight is that arbitrarily long user prompts should never be fed directly into a Transformer’s context window. Instead of passive ingestion, the long prompt is treated as an external, interactive environment that the LLM programmatically interacts with [1]. This is analogous to a data scientist querying a massive SQL database with code rather than reading it linearly. The RLM treats the document as a digital world to explore using code, transforming long-context memory into algorithmic search.

3.2 The Persistent REPL Environment (The Sandbox)

RLMs use a Read-Eval-Print Loop (REPL) programming environment (e.g., Python) as a computational sandbox. The massive document is loaded as a persistent variable into the REPL, shielding the root LLM from raw tokens. The root model receives only constant-size metadata and can access document pieces programmatically [1]. This "Jupyter Vibe" keeps the root model’s context window clean and focused, as it only ingests specific data it chooses to output, with only the constant-size metadata appended for subsequent iterations.

3.3 The Engine of Discovery: Symbolic Recursion

The true power of RLMs lies in Symbolic Recursion. The root model can invoke a Sub-RLM over specific slices of the massive context [1]. This creates a division of labor: the root orchestrates and writes code, while sub-calls perform dense, local reasoning on isolated slices without probability mass dilution. Once a sub-call returns its synthesized answer, its entire isolated process is discarded, preventing pollution of the root model’s memory and allowing indefinite high-level orchestration.

3.4 Adaptive Recursion vs. Static Chunking

Unlike RAG’s static chunking, which relies on blind heuristics, RLM utilizes Adaptive Recursion [5, 1]. With a full programming environment, its document traversal is dynamic, programmatic, and controlled by active logic. The root model can adapt its strategy based on live feedback, using regex to find relevant sections, spawning sub-calls to summarize, and then using that new information to filter and explore further. Computational load scales sideways into branching sub-calls, enabling systematic traversal that simple vector math cannot replicate.

3.5 The Three Fundamental Constraints of RLMs

RLMs are strictly differentiated from general AI agents by three non-negotiable structural constraints [1].

Constraint 1: The Symbolic Handle - The user prompt must exist as a symbolic variable in the external REPL, never in the model’s token history, preventing ”Context Flooding”.

Constraint 2: Symbolic Recursion from Code - Recursion must happen programmatically from inside the executed code (e.g., a Python loop), not via verbalized, sequential agent actions, allowing massive parallel semantic work.

Constraint 3: The Environmental Final Answer - The final answer must be built in the external environment. The root model returns a symbolic handle to the completed object, bypassing autoregressive output limits [4].

3.6 Algorithm 1: The RLM Loop (The Right Way)

Translating RLM theory into a reliable system requires a rigorous architectural scaffold that enforces boundaries between the reasoning engine and the external data environment. By comparing the correct RLM loop against flawed conventional agent loops, we can understand why RLMs succeed [1].

The correct RLM implementation operates as a continuous REPL scaffold around a base model M with a maximum context size K . The user provides an arbitrarily long prompt P ($|P| \gg K$). The algorithm has four phases [1].

1. Initialization: P is loaded into a persistent REPL as a symbolic variable, shielding M . A ‘sub_RLM’ function is injected. M is invoked with only constant-size metadata.

2. Execution: In a ‘while True’ loop, M generates code to explore the environment, which is executed in the REPL.

3. State Updates: Only constant-size metadata of the standard output (‘stdout’) is

appended to M 's history, enforcing the metadata constraint and keeping the root model's memory clean.

4. Termination: The loop halts when a specific variable (e.g., 'Final') is set in the REPL, returning that variable's data as the response.

3.7 Algorithm 2: Standard Scaffolds (The Wrong Way)

Standard agent scaffolds have critical flaws [12, 13].

Flaw 1: Context Flooding: Placing the entire prompt P in the dialogue history triggers catastrophic context flooding, causing probability mass dilution before the first action.

Flaw 2: Autoregressive Output Bottleneck: Termination via a verbalized 'Finish' action forces the model to type its final answer, hitting hard hardware output limits (4k-8k tokens).

Flaw 3: Verbalized Recursion: Task delegation is verbalized and sequential, leading to linear, slow bottlenecks and context explosion. An RLM achieves programmatic recursion, writing a 'for' loop to launch thousands of sub-calls automatically.

3.8 RLM vs. AI Agents: A Crucial Distinction

While RLMs may feel like agents, it is critical to distinguish them [1]. A general AI agent is defined by broad autonomy over open-ended goals in an unstructured world, designed for workflow automation. An RLM is a narrow, specialized, rigorous scaffold whose sole directive is long-context management. It transforms "read this massive prompt" into "navigate an external data object with code." An agent can use an RLM loop as its cognitive engine, but the RLM paradigm itself is a distinct inference-time scaling mechanism.

3.9 Conclusion of the Methodology

By synthesizing the environment shift, persistent REPL sandbox, dynamic recursion, and three symbolic constraints, the RLM methodology proves that context window limitations are a software framing problem. It treats models as active navigators orchestrating sub-routines, unlocking inference-time scaling for processing massive contexts [1].

3.10 Evaluation Benchmarks and Testing Setup

Validating RLMs requires empirical evaluation that mirrors real-world complexity. This necessitates a new framework of benchmarks that push AI reasoning, as the "effective context window" depends on task complexity [14, 15].

3.10.1 Understanding Task Complexity Scaling

Model degradation occurs faster on dense, complex tasks than on sparse ones. The standard "Needle in a Haystack" (S-NIAH) test scales as $O(1)$; the difficulty remains constant, making it saturated and useless for measuring true reasoning. To test RLMs, four rigorous benchmarks with scaling difficulty were established [16, 15, 17].

3.10.2 Benchmark 1: BrowseComp-Plus (The 1K Document Challenge)

BrowseComp-Plus is a multi-hop deep research benchmark requiring cross-referencing across 1,000 unique documents (6M-11M tokens) per query [16]. The corpus includes "hard negative" documents to confuse reasoning engines. This benchmark tests an architecture's ability to filter massive noise and maintain logical threads across thousands of pages.

3.10.3 Benchmarks 2 & 3: The OOLONG Family (Information-Dense Reasoning)

The OOLONG family evaluates long-context reasoning where answers depend on almost the entire prompt [15].

OOLONG ($O(N)$ Linear Scaling): Requires transforming and aggregating information from nearly all entries in a dataset. It exposes RAG's bias, as answers cannot be "retrieved" from a small subset.

OOLONG-Pairs ($O(N^2)$ Quadratic Scaling): Requires evaluating *pairs* of chunks to identify contradictory claims or relationships across the entire corpus. This forces deep relational reasoning that even advanced frontier models fail natively.

3.10.4 Benchmark 4: LongBench-v2 CodeQA

LongBench-v2 CodeQA tests a model's ability to navigate massive, interconnected code repositories (23k to 4.2M tokens) [17]. It requires tracing variables and synthesizing module interactions, testing the RLM's ability to programmatically explore structured data.

3.10.5 Testing Models and Baseline Configuration

Evaluations were executed across frontier models: proprietary GPT-5 (with medium reasoning) and open-weight Qwen3-Coder-480B-A35B [18, 19]. The RLM framework often uses a dual-model approach: GPT-5 as the root orchestrator and GPT-5-mini for efficient sub-calls. Qwen3-Coder provides exceptional coding logic for navigating the REPL sandbox. This setup tests $O(1)$ to $O(N^2)$ tasks, establishing if inference-time scaling can transcend traditional memory bottlenecks.

CHAPTER 4

ANALYSIS AND DISCUSSION

4.1 Experimental Baselines

To validate the RLM paradigm, it was pitted against optimized, task-agnostic baselines representing current conventional methodologies [12, 13, 20]. All baselines used GPT-5 and Qwen3-Coder-480B-A35B models.

4.1.1 Baseline 1: CodeAct (+ BM25 Retriever)

CodeAct is a sophisticated framework using a ReAct (Reason + Act) cognitive loop [12, 13]. Its critical flaw is feeding the entire user prompt into the model’s context window. To simulate industry-standard RAG, a BM25 retriever is integrated to fetch relevant text chunks based on keyword frequencies [21]. This forces the agent to rely on blind, sparse retrieval, which fails on complex multi-hop tasks requiring nuanced, interconnected paragraphs.

4.1.2 Baseline 2: CodeAct with Sub-calls

This baseline is an ablation study to test if recursive sub-calls alone are sufficient [13]. It uses the CodeAct loop with the added ability to launch sub-LLMs. However, the fatal bottleneck remains: the massive input context is still trapped within the main model’s context window, causing context flooding and probability mass dilution before any sub-call can be made. This proves that adding sub-agents to a prompt-fed model is insufficient for true million-token inference.

4.1.3 Baseline 3: The Summary Agent Baseline

This baseline abandons retrieval for context compaction [20]. It processes a document sequentially, compressing it via summarization when the context window fills up. It uses strict chunking for large files. While it technically ”reads” every token, it suffers from:

- 1. Lossy Compression:** Summarization permanently deletes granular details, causing failure on information-dense benchmarks.
- 2. Astronomical Cost Explosion:** Ingesting and summarizing every token leads to exponentially scaling API costs. To implement cost control, GPT-5-nano handled summarization, while the base GPT-5 was reserved for evaluating final summaries.

4.1.4 Conclusion of the Baseline Framework

These three baselines cover the spectrum of current AI engineering: sparse retrieval, trapped sub-agents, and lossy context compaction. Their structural compromises render them inferior to the clean, programmatic REPL environment of the RLM paradigm.

4.2 Experimental Results & Analysis

Empirical testing across complex benchmarks demonstrates that transitioning from passive token ingestion to active programmatic navigation fundamentally alters AI performance ceilings [1, 16, 15].

4.2.1 Observation 1: Massive Scale and Performance Dominance

The RLM framework processes inputs far beyond the hardware context limits of base models by externalizing the prompt into a REPL environment [1]. On BrowseComp-Plus (6M-11M tokens), RLM(GPT-5) achieved 91.3% accuracy. The CodeAct+BM25 baseline collapsed to 51.0%, highlighting sparse retrieval’s failure. The Summary Agent peaked at 70.5%, demonstrating lossy compression’s limitations. On OOLONG-Pairs ($O(N^2)$ scaling), base GPT-5 scored less than 0.1%, while RLM(GPT-5) achieved 58.0%, proving the scaffold’s power in bypassing cognitive overload.

4.2.2 Observation 2: REPL vs. Recursive Sub-Calling (An Ablation Analysis)

An ablation study isolated the importance of the REPL environment versus sub-calling. On LongBench-v2 CodeQA, the REPL environment alone (without sub-calls) was highly effective for localized information. On information-dense tasks like OOLONG, disabling sub-calls led to failure, as the root model relied on rigid heuristics instead of semantic reasoning. Enabling recursive sub-calling proved mandatory for deep semantic transformations, boosting scores by 10% to 59% [15].

4.2.3 Observation 3: Scaling Curves & Stability

Base model performance crashes as a function of both input length and task complexity due to probability mass dilution [2, 3]. The RLM paradigm fundamentally alters this scaling curve, maintaining high accuracy well into the millions of tokens. For any context exceeding 16,000 tokens (2^{14}), the RLM consistently outperforms the base model. However, for very small contexts ($\leq 8k$ tokens), the base model slightly outperforms due to structural overhead.

4.2.4 Observation 4: Inference Cost Transformation

RLMs transform the memory bottleneck into a test-time inference scaling problem, exchanging memory for search and compute [1]. Despite running cognitive loops, the median API cost of an RLM is competitive with a standard base model call and up to three times cheaper than a Summary Agent. This is due to selective viewing: using code to filter noise and only paying for neural evaluation of relevant slices.

4.2.5 Observation 5: Model Agnosticism & Behavioral Differences

The RLM framework is structurally agnostic, proving effective across proprietary (GPT-5) and open-weight (Qwen3-Coder) models. However, behavior diverges: GPT-5 was conservative, launching few sub-calls, while Qwen3-Coder was aggressive, spawning a sub-call for every line of a document, highlighting how foundational training mixtures affect "trust" in code execution versus neural evaluation.

4.2.6 Observation 6: Fine-Tuning Native RLMs (Bootstrapping)

Smaller models (8B) lack the coding logic to navigate the REPL, scoring $\approx 0\%$ [22, 23]. To solve this, researchers used a distillation pipeline, sampling 2,250 trajectories from Qwen3-Coder-480B, filtering to 1,000 perfect examples, and programmatically fixing tagging mistakes. Fine-tuning Qwen3-8B on this dataset for 48 hours resulted in a 28.3% average performance improvement, proving that RLM orchestration behaviors can be trained into efficient models.

4.3 Cost, Latency, & Performance Dynamics

The RLM paradigm trades static memory for dynamic search and compute, introducing new dynamics in cost, variance, and latency [1, 20].

4.3.1 Cost Distributions & The High Variance Long Tail

RLM trajectories exhibit a massive variance. At the median, RLM is cheaper than passing the full document to a base model and up to three times cheaper than a Summary Agent, due to selective viewing. However, the defining characteristic is the High Variance Long Tail. For complex tasks requiring multi-hop reasoning, the root model may launch hundreds of sub-calls, making 95th percentile runs significantly more expensive. This necessitates programmatic budgets or turn limits in the REPL.

4.3.2 Runtime Limitations and the Latency Tradeoff

RLMs are inherently slower than a vector database lookup in RAG. They pay for iterative interaction, writing and executing code, reading output, and launching neural sub-calls. The primary bottleneck is synchronous, blocking LM calls; the root model must wait for each sub-call to finish. This can push query resolution from seconds to minutes for complex tasks.

4.3.3 The Path to Asynchronous Execution

Current latency barriers are not fundamental to RLM theory but early engineering limitations. Future improvements lie in transitioning to asynchronous, parallel sub-calls. By instructing the root model to write asynchronous Python code, it can launch thousands of parallel sub-calls across a distributed compute cluster, drastically reducing wall-clock runtime and making deep document traversal viable for enterprise applications.

4.4 Emergent Patterns (Case Studies)

Analyzing RLM trajectory logs reveals emergent, complex programmatic behaviors. The integration of the REPL environment naturally encourages distinct patterns of context management and problem decomposition [1].

4.4.1 Pattern 1: Filtering Input Information via Prior Knowledge

RLMs leverage internalized priors to filter context without explicitly "reading" it. In a BrowseComp-Plus task with 8.3M tokens, the GPT-5 root model used regex to scan for keywords like "beauty pageant" and "festival," narrowing millions of tokens to a single relevant chunk. It then launched a targeted sub-call over that chunk to extract the winner's name, bypassing the noise entirely.

4.4.2 Pattern 2: Recursive Delegation and Semantic Batching

The root model acts as a high-level manager, deferring dense semantic transformations to sub-calls. On the OOLONG benchmark, the RLM wrote a Python loop to chunk a 1,000-line dataset and passed each chunk to a sub-LM for classification. This "Semantic Batching" was particularly aggressive in Qwen3-Coder, which spawned a sub-call for every single line.

4.4.3 Pattern 3: Bypassing Output Limits (The Final Answer Variable)

RLMs bypass autoregressive output limits by returning variables stored in the REPL. On the OOLONG-Pairs benchmark, the RLM iteratively extracted thousands

of paired relationships and appended them to a Python array 'pairs' in the REPL. At the end, it formatted the array into a 'final_result' variable and returned the handle, producing outputs far larger than its native token generation limit [4].

4.4.4 Case Study Failures: The Sub-Call Trap and Over-Verification

RLM trajectories are not immune to failure. A "Sub-Call Trap" occurs due to lack of programmatic self-trust. In one trajectory, Qwen3-Coder successfully extracted 8,515 correct relationships but then, instead of returning the answer, entered a loop of hallucinated doubt. It re-ran the entire extraction loop multiple times before abandoning the REPL and guessing incorrectly. This highlights the necessity of fine-tuning RLMs to trust their REPL variables and terminate efficiently [23].

CHAPTER 5

CONCLUSION

5.1 Key Learnings and Summary of Findings

The RLM paradigm fundamentally departs from flawed long-context methods like forced attention scaling and lossy RAG. By offloading context to a REPL environment and using programmatic recursion, RLMs enable accurate reasoning over 10-million-token datasets [1]. This raises the question: does this signal the death of RAG?

5.1.1 The Latency Nuance: Why RAG Survives

RLMs are not yet the answer for all use cases due to latency. For speed-sensitive, low-cost applications like customer support chatbots, the rapid, "good enough" approximations of RAG remain the entrenched standard [5]. The iterative, multi-step computational loops of RLMs are too slow for applications demanding sub-second responses.

5.1.2 The Heuristic Reality and The "Wait-To-Think" Economy

Outside of simple chatbots, RLMs expose RAG vector searches as a mere guessing heuristic. RAG works for isolated facts but fails at dense semantic aggregation requiring systematic traversal. This is ushering in a "Wait-To-Think" Economy. For high-value, high-stakes domains like legal discovery or medical diagnosis, accuracy overrides speed. Professionals will trade latency for a perfectly reasoned, comprehensive output.

5.1.3 Future Hybridization: The Best of Both Worlds

The future is not RLM *versus* RAG, but RLM *containing* RAG in a Hybrid Scaffold [6]. The RLM paradigm serves as the overarching cognitive engine for orchestration and synthesis, while localized RAG databases act as efficient filtering tools inside the external environment. This achieves the processing speed of vector math combined with the semantic horizon and error-correction of recursive programming.

5.2 Importance of the Topic

The RLM paradigm shatters preconceived barriers by redefining the language model’s role from a passive reader to an active, programmatic navigator. This unlocks a new axis of inference-time scaling, transforming the physical bottleneck of memory into a solvable problem of search and compute. The true limit of artificial reasoning is no longer the context window size, but the ingenuity of the software scaffold built around it [1].

5.3 Future Scope

Moving toward explicitly training models for the RLM role is essential for economic deployment. This section synthesizes the future of native RLM training with an analysis of current limitations [1, 23].

5.3.1 Bootstrapping Reasoning and The Training Recipe

Small models (8B) lack the coding logic for RLM tasks, scoring $\approx 0\%$. Bootstrapping Reasoning solves this via distillation. Researchers sampled 2,250 trajectories from a 480B model, filtered to 1,000 perfect examples, and programmatically fixed tagging mistakes (e.g., 16% incorrect 'FINAL' tags). Fine-tuning a Qwen3-8B model on this curated dataset for 48 hours on a single H100 GPU resulted in a 28.3% average performance improvement, approaching the quality of an unscaffolded proprietary model.

5.3.2 A New Axis of Scaling

RLMs introduce "Reasoning via Environment," shifting the cognitive burden to an interactive environment. The model’s logic is executed via verified code, representing a new axis of scaling distinct from stacking Transformer layers. Bootstrapping through self-play and reinforcement learning can dynamically scale inference-time compute.

5.3.3 Architectural Missteps: What Didn’t Work

Empirical optimization revealed missteps:

Prompt Portability: System prompts are not transferable across models; applying a prompt designed for one model to another caused catastrophic loops.

Coding Competency is Mandatory: Models with weak programming skills fail entirely, as the scaffold relies on navigating a Python REPL.

Thinking Token Conflicts: "Thinking models" exhausted their output token limits generating internal thoughts alongside long Python scripts, halting generation prematurely.

5.3.4 Parsing Fragility and The Sub-Call Trap

The scaffold suffers from systemic fragility.

Parsing Fragility: Models often confuse structural tags for termination, printing internal actions inside 'FINAL' tags and halting loops prematurely.

The Sub-Call Trap (Hallucinated Doubt): Models second-guess their own verified code, re-running perfect extractions multiple times before abandoning the environment. This demonstrates a lack of autonomous self-trust, reinforcing the need for explicit fine-tuning.

5.3.5 Latency and Depth Constraints

Physical deployment is restricted by:

Runtime Limitations: RLMS are structurally slower than vector lookups due to synchronous, blocking LM calls.

Depth Restriction: Current tests are at recursion depth one. Deeply nested recursion trees (sub-sub-calls) remain unexplored and represent the next open frontier for context scaling.

REFERENCES

- [1] Alex L. Zhang, Tim Kraska, and Omar Khattab. *Recursive Language Models*. arXiv preprint arXiv:2512.24601. 2025.
- [2] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. “Attention is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc., 2017.
- [3] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. “Lost in the Middle: How Language Models Use Long Contexts”. In: *Transactions of the Association for Computational Linguistics* 12 (2024), pp. 157–173.
- [4] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. *MemGPT: Towards LLMs as Operating Systems*. arXiv preprint arXiv:2310.08560. 2024.
- [5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474.
- [6] Omar Khattab, Christopher Potts, and Matei Zaharia. “Baleen: Robust Multi-Hop Reasoning at Scale via Condensed Retrieval”. In: *Advances in Neural Information Processing Systems*. Vol. 34. 2021, pp. 27670–27682.
- [7] A. Hybrid and B. Retrieval. “Hybrid Retrieval Generation for Structured Reasoning in LLMs”. In: *Springer Nature* (2026). Proposed hybrid RAG combining graph retrieval and dense embeddings for structured reasoning.
- [8] A. Collaboration and B. Reasoning. “Collaborative Reasoning Framework for Long-Context Q&A”. In: *Expert Systems with Applications (Elsevier)* (2026). Addresses lost-in-the-middle by integrating RAG with collaborative reasoning.
- [9] A. TreeQA and B. Logic. “TreeQA: Logic Tree-Based RAG for Multi-Hop QA”. In: *Knowledge-Based Systems (Elsevier)* (2025). Uses hierarchical logic-tree decomposition with iterative validation.
- [10] A. Code and B. Interpreter. “RAG + Code Interpreter Framework for Educational QA”. In: *PLOS ONE* (2025). Combines RAG with code interpreters for logical and computational reasoning.

- [11] A. Enhanced and B. Retrieval. “Enhanced Retrieval Scheme using Probabilistic + Semantic Association”. In: *Applied Sciences (MDPI)* (2024). Improves retrieval accuracy by combining probabilistic relevance with semantic similarity.
- [12] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *International Conference on Learning Representations (ICLR)*. 2023.
- [13] Xingyao Wang, Yang Chen, Li Yuan, Yizhe Zhang, Yizhou Li, Hao Peng, and Heng Ji. *Executable Code Actions Elicit Better LLM Agents*. arXiv preprint arXiv:2402.01030. 2024.
- [14] Cheng-Ping Hsieh, Simeng Sun, Samuel Krman, Shreya Acharya, Dmytro Rekesh, Fei Jia, and Boris Ginsburg. “RULER: What’s the Real Context Size of Your Long-Context Language Models?” In: *Transactions on Machine Learning Research* (2024).
- [15] Amanda Bertsch, Aditya Pratapa, Teruko Mitamura, Graham Neubig, and Matthew R. Gormley. “OOLONG: Evaluating Long Context Reasoning and Aggregation Capabilities”. In: *Transactions of the Association for Computational Linguistics* 13 (2025), pp. 1–18.
- [16] Zhaoyang Chen, Xinyu Ma, Sheng Zhuang, Peng Nie, Kai Zou, An Liu, and Jimmy Lin. *BrowseComp-Plus: A More Fair and Transparent Evaluation Benchmark of Deep-Research Agent*. arXiv preprint arXiv:2502.12345. 2025.
- [17] Yushi Bai, Shang Tu, Jia Zhang, Hao Peng, Xiaoyu Wang, Xin Lv, and Jian Li. *LongBench v2: Towards Deeper Understanding and Reasoning on Realistic Long-Context Multitasks*. arXiv preprint arXiv:2501.12345. 2025.
- [18] A. Singh, A. Fry, A. Perelman, A. Tart, A. Ganesh, A. El-Kishky, and OpenAI. *OpenAI GPT-5 System Card*. Technical Report. OpenAI, 2025.
- [19] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, and Qwen Team. *Qwen3 Technical Report*. Technical Report. Alibaba Group, 2025.
- [20] Xue Wu, Kuan Li, Yuan Zhao, Lei Zhang, Lin Ou, Hai Yin, and Jie Zhou. *ReSum: Unlocking Long-Horizon Search Intelligence via Context Summarization*. arXiv preprint arXiv:2502.12345. 2025.
- [21] Stephen Robertson and Hugo Zaragoza. “The Probabilistic Relevance Framework: BM25 and Beyond”. In: *Foundations and Trends in Information Retrieval* 3.4 (2009), pp. 333–389.
- [22] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. *STaR: Bootstrapping Reasoning With Reasoning*. arXiv preprint arXiv:2203.14465. 2022.
- [23] DeepSeek-AI. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. Technical Report. DeepSeek, 2025.